FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO - FECAP

Bacharelado em Ciência da Computação

Sistemas Operacionais e Computação em nuvem - Rodrigo Da Rosa

Estudantes:

- André Gregório dos Santos – RA: 24026489
- Guilherme Reis Fogolin de Godoy – RA: 24026241
- Pedro Henrique Nascimento Lemos – RA: 23025380
- Yan Ramos Cezareto – RA: 24026005

Turma: 5NACOMP_S

Entrega 01 - Sistemas Operacionais e Computação em nuvem

1. Objetivo

O objetivo deste projeto foi realizar a configuração inicial de uma máquina virtual em nuvem, preparando o ambiente para o desenvolvimento de aplicações Python/Fast API através da manipulação de pacotes, arquivos e diretórios via terminal Linux.

2. Configuração do Sistema e Instalação de Ferramentas

A primeira etapa consistiu na atualização dos repositórios do sistema e na instalação do gerenciador de pacotes do Python (pip), essencial para a gestão de bibliotecas do projeto.

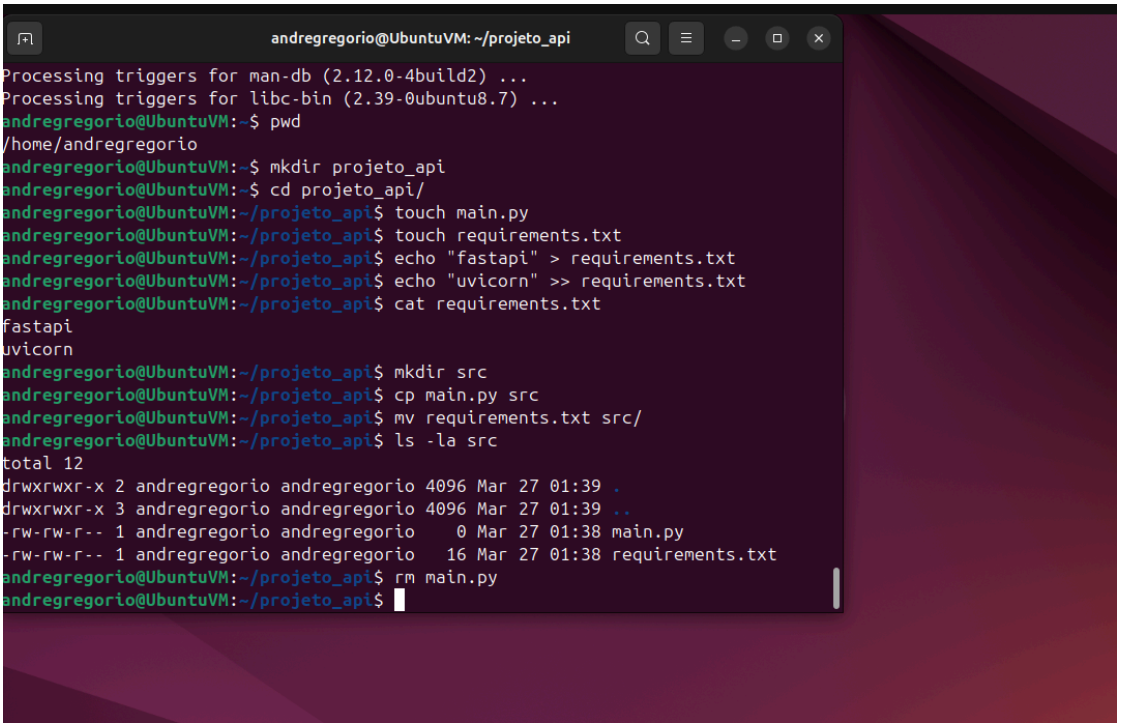
```
andregregorio@UbuntuVM: ~/projeto_api
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

andregregorio@UbuntuVM:~$ sudo apt update
[sudo] password for andregregorio:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
41 packages can be upgraded. Run 'apt list --upgradable' to see them.
andregregorio@UbuntuVM:~$ sudo apt install python3-pip -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential bzip2
  dpkg-dev fakeroot g++ g++-13 g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu
  gcc gcc-13 gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu javascript-common
  libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl
  libasan8 libbinutils libcc1-0 libctf-nobfd0 libctf0 libdpkg-perl
  libexpat1-dev libfakeroot libfile-fcntllock-perl libgcc-13-dev libgprofng0
```

Nesta etapa, foram executados os seguintes comandos:

1. sudo apt update: Sincroniza os índices de pacotes das fontes configuradas para garantir que o sistema conheça as versões mais recentes disponíveis.
2. sudo apt install python3-pip -y: Instala o instalador de pacotes para Python 3. O parâmetro -y foi utilizado para automatizar a confirmação da instalação de todas as dependências necessárias (como build-essential, gcc, etc).

3. Manipulação de Arquivos e Estruturação do Projeto

Nesta etapa, o foco foi a organização lógica do ambiente de desenvolvimento. Através do terminal, foi criada uma estrutura de diretórios seguindo boas práticas de desenvolvimento de software, permitindo o isolamento do código-fonte e das dependências.



Procedimentos Realizados:

- **Identificação e Criação do Workspace:** O processo iniciou-se com o comando pwd para confirmar o diretório atual do usuário. Em seguida, utilizou-se o comando mkdir projeto_api para criar a pasta raiz do projeto, seguida pelo comando cd para ingressar no novo diretório.
- **Geração de Arquivos de Configuração:** Foram criados os arquivos essenciais utilizando o comando touch. Para a gestão de dependências, o arquivo requirements.txt foi preenchido dinamicamente via terminal:
  - O comando echo "fastapi" > requirements.txt foi utilizado para inserir a primeira biblioteca, sobrescrevendo o arquivo.

    - O comando echo "uvicorn" >> requirements.txt adicionou a segunda biblioteca ao final do arquivo, demonstrando o uso correto de operadores de redirecionamento de saída.
    - A integridade do arquivo foi verificada com o comando cat, que exibiu o conteúdo diretamente no prompt.
- **Organização e Refatoração da Estrutura:** Para separar o código-fonte dos arquivos de configuração da raiz, foi criado um subdiretório chamado src (mkdir src). A organização prosseguiu com:
    - Cópia de Segurança: O arquivo main.py foi copiado para a pasta de destino usando cp.
    - Movimentação de Arquivos: O arquivo de requisitos foi movido permanentemente para dentro de src através do comando mv.
    - Auditoria de Diretório: Utilizou-se o comando ls -la src para listar de forma detalhada o conteúdo da pasta, verificando permissões, proprietário e tamanho dos arquivos.
- **Limpeza de Resíduos:** Para finalizar a estruturação e evitar redundâncias, o arquivo original main.py (que permanecia na raiz após a cópia) foi removido utilizando o comando rm, garantindo que apenas a versão organizada dentro do diretório src fosse mantida**.**

4. Dificuldades Encontradas e Soluções Aplicadas

- **Gerenciamento de Privilégios:** Durante a instalação do pip, o sistema solicitou privilégios de administrador. A solução foi o uso correto do comando sudo seguido da autenticação do usuário.
- **Concatenação de Arquivos:** Ao criar o requirements.txt, houve o cuidado de utilizar o operador >> para o segundo pacote, evitando que a primeira entrada ("fastapi") fosse apagada.
- **Organização de Arquivos:** Inicialmente, os arquivos foram criados na raiz. A solução aplicada foi a utilização dos comandos mv e cp para organizar o código dentro de uma pasta src, seguindo boas práticas de desenvolvimento.

5. Conclusão

A configuração foi concluída com sucesso. Os comandos utilizados permitiram a criação de um ambiente estruturado e pronto para execução de serviços web. O domínio do terminal Linux demonstrou-se fundamental para a agilidade no gerenciamento de recursos em servidores de nuvem.